Technical Report — FastSpeech 2 for English and Telugu

Nov 7, 2026

1. Introduction

We built two text-to-speech (TTS) systems, one for English and one for Telugu. Each takes a sentence as text and produces spoken audio. Both use FastSpeech 2, a well-known TTS model [1], starting from a public implementation on GitHub (ming024/FastSpeech2). Each model was trained on about 5 hours of speech and checked on a separate 1 hour.

How the system works

1. **Text to phones.** The sentence is turned into phones, the basic sounds of the language (for example, "cat" becomes K AE T).
2. **FastSpeech 2.** The model predicts how long each phone lasts (duration), how high the voice is (pitch) and how loud it is (energy). From these it draws a mel-spectrogram: a picture of the sound over time.
3. **Vocoder.** A second model, HiFi-GAN, turns that picture into an audio waveform.

Terms used in this report

Term	Meaning
Phone	a single speech sound, such as /k/ or /a/
G2P	grapheme-to-phoneme: converting written text into phones
Mel-spectrogram (mel)	an image of audio: time on one axis, frequency on the other
Variance adaptor	the part of FastSpeech 2 that predicts duration, pitch (F0) and energy for each phone
Vocoder	the model that turns a mel-spectrogram into sound
Alignment	finding where each phone starts and ends in a recording
Overfitting	a model memorizing its training data instead of learning to handle new data

Term	Meaning
Checkpoint	a saved copy of a model's weights at a given training step

Main results

- Both models produce intelligible speech for all ten test sentences.
- Both models stopped improving on new data after about 15,000–25,000 training steps, so we used the step-25,000 checkpoint.
- The first outputs sounded muffled. We traced this to the FastSpeech 2 output, not the vocoder, and fixed most of it by retraining the vocoder on FastSpeech 2's own output. Listeners preferred the retrained vocoder in 13 of 15 comparisons.
- The main remaining errors come from turning text into phones: for example, "conduct" is always read as the noun, even where the sentence needs the verb.

2. Implementation

We kept FastSpeech 2's design exactly as in the public implementation and changed only what was needed to support Telugu and to run on current software.

Parts of the model

Part	What it does
Encoder	reads the phone sequence and works out how neighbouring phones affect each other
Variance adaptor	predicts duration, pitch and energy for each phone
Length regulator	stretches the sequence so each phone lasts its predicted duration
Decoder and postnet	draw the mel-spectrogram and then sharpen it
Speaker embedding (English only)	tells the model which of the 115 English speakers to imitate
HiFi-GAN vocoder	turns the mel-spectrogram into audio

During training the model learns from the real duration, pitch and energy of each recording, and is scored on how close its mel-spectrogram is to the real one. One simplification from the original paper: we predict one pitch and one energy value per

phone, instead of a value for every short time frame. The public code does not include the paper's wavelet-based pitch method, it is more stable with only 5 hours of data, and it lets the duration, pitch and energy plots share the same phone axis.

Changes to the public code

File	Change	Reason
`text/symbols.py`	added the 46 Telugu phones	the model silently ignores any phone it doesn't know
`preprocessor/preprocessor.py`	treat empty segments in alignment files as pauses	the newer aligner marks pauses this way; ignoring them shifted the timing of everything after the first pause
`utils/tools.py`	convert pitch and energy values to 32-bit numbers	mixing 32- and 64-bit numbers crashed training
`audio/stft.py`, `audio/audio_processing.py`	updated calls to the audio library	the library changed its interface
`utils/model.py`	updated how saved models are loaded	newer PyTorch blocks the old way by default
`config/`	new settings for each language (Section 5)	each language needs its own paths and settings

We also wrote new code: text processing for both languages (`frontend/`), the fixed train/validation split, inference and plotting, and simple `train.py` , `preprocess.py` and `inference.py` scripts for each language. The vocoder training code (jik876/hifi-gan) received similar compatibility fixes.

3. Data

For each language we picked about 5 hours of recordings for training and 1 hour for validation (checking the model on speech it has not trained on). The choice is random but repeatable (fixed seed 42), and a check in the code confirms that no recording appears in both sets.

	English	Telugu
Dataset	Svarah (Indian-English speech, recorded for speech recognition)	IndicTTS Telugu (studio recordings for TTS)
Speakers used	115	1 (male)
Training	3,320 clips, 5.12 h	2,341 clips, 5.00 h
Validation	655 clips, 0.97 h	468 clips, 1.00 h
Speech after removing silence at the start and end	4.77 h + 0.90 h	4.27 h + 0.85 h

English. Svarah has many speakers but only about 4 minutes from each, so no single speaker had enough audio. We therefore trained one model on all speakers and gave it a speaker ID, so it can tell the voices apart instead of blending them. The dataset has no speaker ID column, so we built one from the eight details recorded about each person (gender, age group, home state and so on). This gave 115 speakers, close to the 117 the dataset's authors report. We took about 75% of each speaker's clips for training and the rest for validation. Validation therefore tests new sentences from familiar voices, not new voices. This matches how the model is used: it can only speak in a voice it has learned.

Telugu. The female speaker has only 3.8 hours of usable audio, which is not enough, so we used the male speaker (8.1 hours, recorded at higher quality).

Both languages. We kept clips between 1 and 15 seconds long with a valid transcript. A few clips (3 English, 4 Telugu) were dropped later because their alignment or pitch could not be computed.

4. Preparing the data

Audio

All recordings were converted to the same format: one channel, 22,050 samples per second. This is the format the vocoder expects. The English recordings were originally lower quality (16,000 samples per second), and converting them up cannot add the missing detail, so English sounds slightly duller than Telugu.

Text to phones

English. Numbers, times and dates are first written out in words ("10:45 AM" → "ten forty five A M", "12 November 2026" → "twelfth November twenty twenty six"). Each word is then looked up in a pronunciation dictionary. Words not in the dictionary get their phones from a model that guesses pronunciation from spelling (g2p_en). Punctuation becomes a short pause.

Telugu. Telugu is written almost exactly as it is spoken, so a few simple rules convert it to phones:

1. Every consonant carries the vowel "a" unless something removes it.
2. The virama mark (్) removes that vowel, which joins consonants (పట్టు → p a tt tt u).
3. A vowel sign replaces the "a".
4. The anusvara (ం) becomes the nasal sound that matches the next consonant (బలంగా → b a l a ng g aa).
5. The visarga (ః) becomes "h".

Numbers are written out in Telugu words (2026 → రెండు వేల ఇరవై ఆరు, 12వ → పన్నెండవ). Some transcripts used the letter "o" or the digit "0" in place of the anusvara, and these were corrected.

Alignment

To learn durations, the model needs to know where each phone starts and ends in every training recording. We found this with the Montreal Forced Aligner (MFA).

- **English:** MFA's ready-made English model was used. All 3,975 recordings were aligned.
- **Telugu:** no ready-made Telugu model exists, so we trained one on our own Telugu data. 2,805 of 2,809 recordings were aligned.
- **A bug we fixed:** the aligner marks pauses as empty segments, and the original code skipped them. That would have shifted the timing of every sound after the first pause.

5. Training setup

Both models used the same settings, apart from the speaker ID, which only English needs.

Setting	Value
Training steps	100,000
Batch size	16 clips per step

Setting	Value
Learning-rate schedule	warm up for 4,000 steps, then reduce at 60k, 80k and 90k steps
Validation	every 1,000 steps
Model saved	every 5,000 steps
Longest input	1,500 frames, about 17 seconds
Training time	about 3 h 10 min (English), 3 h 30 min (Telugu)
Hardware	one NVIDIA H100 GPU, shared by both runs

Two settings differ from the public code's defaults. The longest input was raised from 1,000 frames (about 11.6 seconds) because our clips go up to 15 seconds. The learning-rate reductions were moved earlier because the defaults (300k+ steps) would never be reached in a 100k-step run.

6. Training and validation loss

The loss measures how far the model's output is from the real recording; lower is better. **Training loss** is measured on the clips the model learns from, and **validation loss** on clips it never sees. If training loss keeps falling while validation loss stops, the model is memorizing rather than learning.

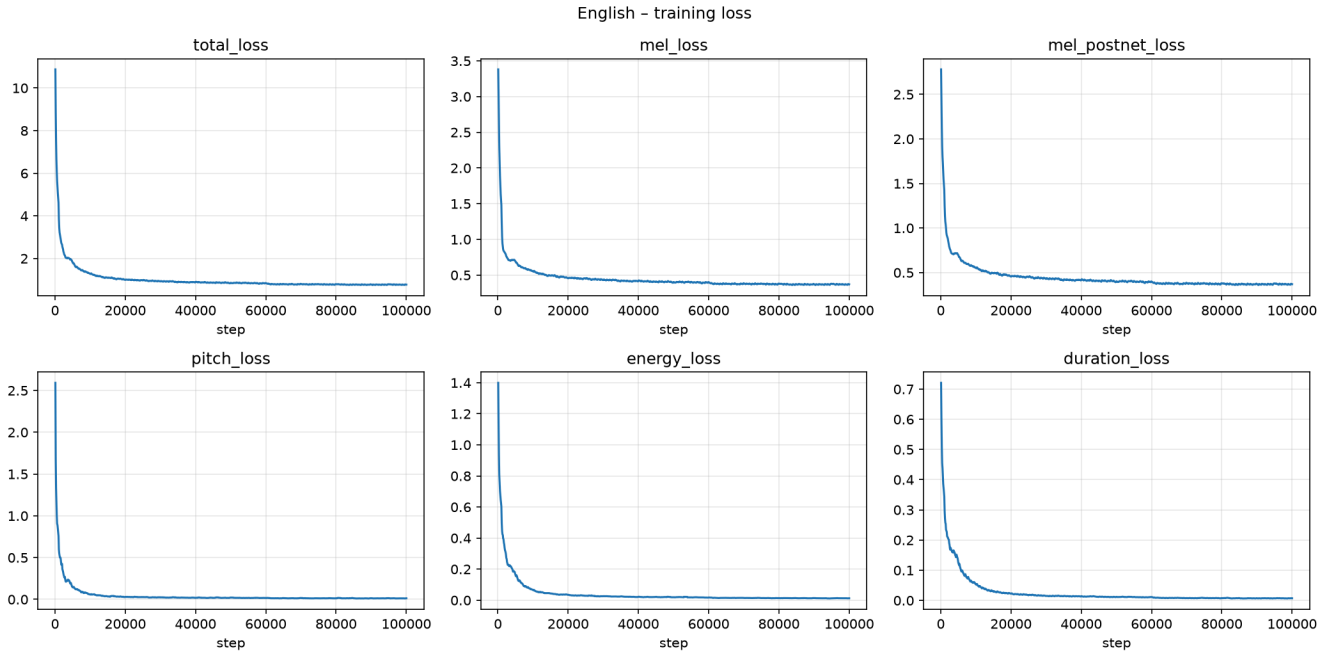


Figure 1. English: training loss

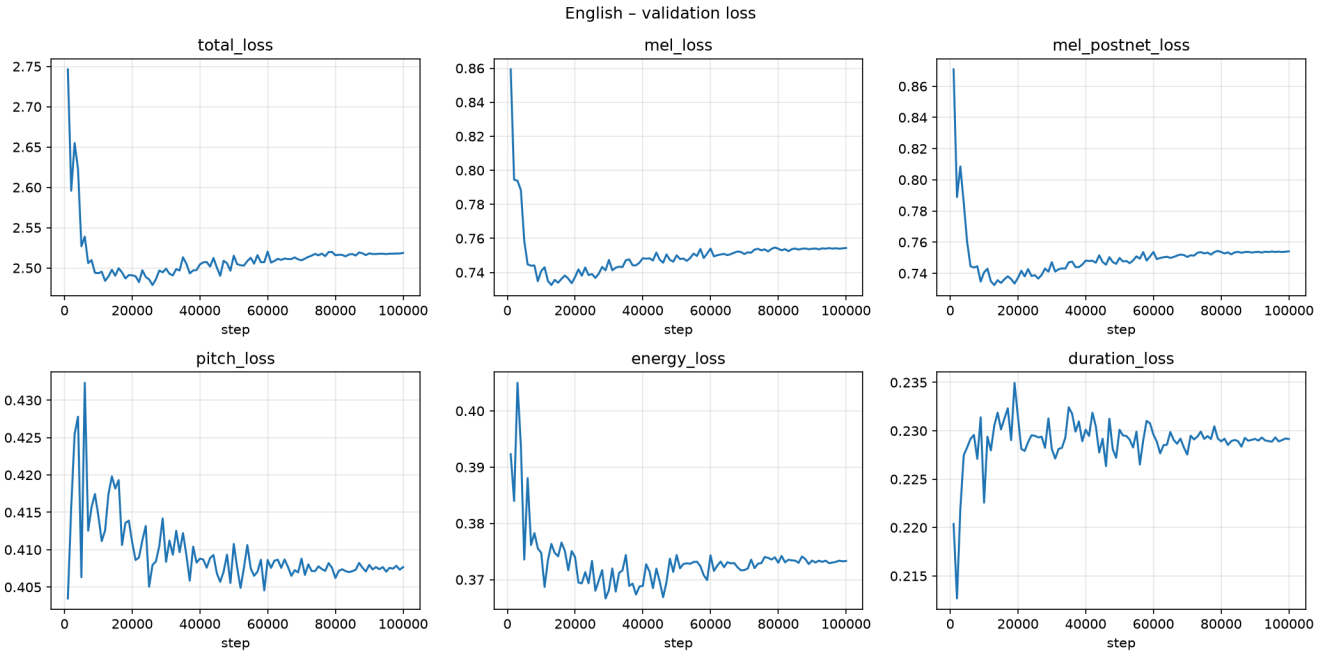


Figure 2. English: validation loss

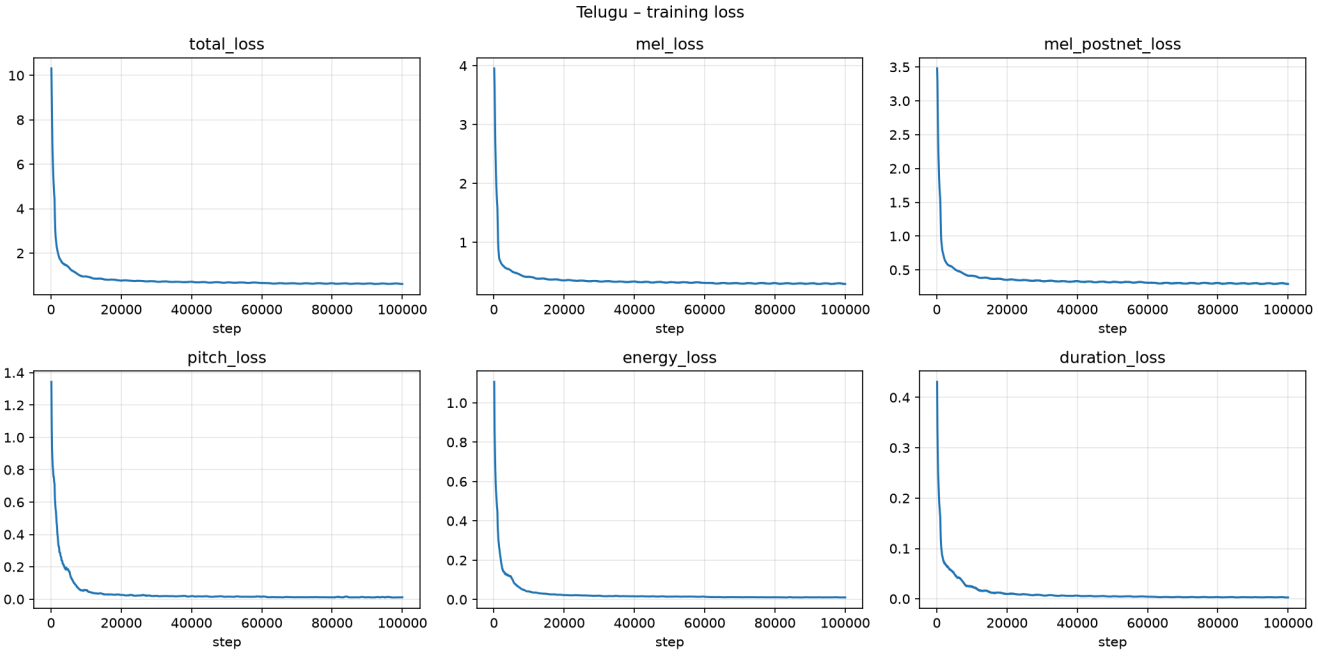


Figure 3. Telugu: training loss

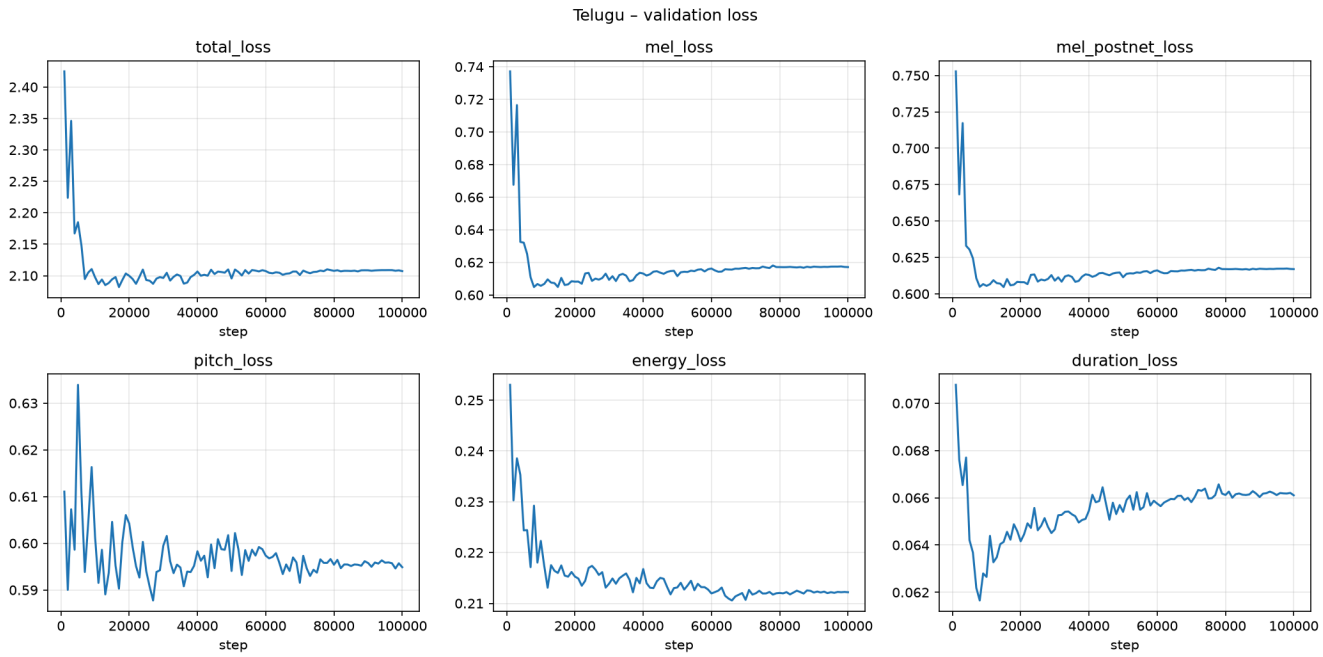


Figure 4. Telugu: validation loss

What happened. Training ran smoothly, with no crashes or sudden jumps. Validation loss improved until about 15,000–25,000 steps and then stayed almost flat, while training loss kept falling until the end at 100,000 steps. In other words, after about 25,000 steps the models were memorizing the training clips. This is expected with only 5 hours of data. We therefore used the model saved at step 25,000, which had the lowest validation loss. A version from step 100,000 did not sound any better.

Loss	English: validation at 25k	English: training at 100k	Telugu: validation at 25k	Telugu: training at 100k
Mel (sound)	0.74	0.36	0.61	0.28
Pitch	0.41	0.02	0.59	0.01
Duration	0.23	0.01	0.07	0.00

What the numbers show

- **The gap between training and validation is large,** especially for pitch and duration. The models predict the pitch and timing of sentences they have memorized very well, and those of new sentences much less well.
- **English timing is harder to predict than Telugu timing** (validation duration loss 0.23 against 0.07). English has 115 speakers who all speak at different speeds, while Telugu has one consistent speaker.

- **Telugu pitch is harder to predict than English pitch** (0.59 against 0.41). The pitch values are scaled differently in each language, so this comparison is only approximate.
- **Validation loss is optimistic.** It is measured while giving the model the real duration, pitch and energy. When generating new speech, the model has to use its own predictions, so the real error is larger.

7. Improving the sound quality

The first audio was understandable but muffled and slightly noisy. We tested where the problem came from, fixed it, and confirmed the fix with listeners.

Finding the cause

We made four versions of the same validation clips. Each version replaces one more real part with a generated one, so the point where the quality drops shows where the problem is.

Version	How it was made	How it sounded
1	the original recording	good
2	the real mel-spectrogram, turned into audio by the vocoder	good
3	FastSpeech 2's mel-spectrogram, given the real duration, pitch and energy	clearly worse
4	FastSpeech 2's mel-spectrogram, using its own predictions	worse

The vocoder handles real mel-spectrograms well (version 2), so it was not the problem. The quality dropped as soon as FastSpeech 2 drew the mel-spectrogram (version 3). Its pictures are blurrier than real ones, because training rewards it for producing an average of possible outputs. The vocoder had never seen such blurry pictures.

The fix: retraining the vocoder

We let FastSpeech 2 draw blurry mel-spectrograms for every training clip, while giving it the real timing so they line up exactly with the real audio. We then retrained the vocoder on these pairs (blurry picture → real audio), so it learned to turn FastSpeech 2's output into cleaner sound. This is a standard technique, also used in the HiFi-GAN [2] and Tacotron 2 [3] papers.

We trained one vocoder per language, starting from the ready-made one, for 28,000 steps. The version from step 20,000 had the lowest validation error in both languages, and later versions got slightly worse (the same memorizing effect as in Section 6), so we used step 20,000.

Listening test

To check the result fairly, we built a small web page that played four versions of each test sentence in a random order, labelled only A to D: the original vocoder and the retrained vocoder at steps 20k, 26k and 28k. Listeners picked the clearest one without knowing which was which.

Version picked	English	Telugu	Total
Original vocoder	1	1	2
Retrained, step 20k	3	1	4
Retrained, step 26k	4	1	5
Retrained, step 28k	2	2	4

The retrained vocoder was chosen in 13 of 15 cases, and the three retrained versions sounded the same. With only two listeners this is a clear trend but not proof, and neither listener speaks Telugu.

8. Results for the test sentences

All ten sentences are understandable. English still has a little background noise and sounds slightly muffled, which comes from its lower-quality training recordings. For each sentence, the plots in `Results/<language>/variance_adaptor_plots/` show the predicted duration, pitch and energy of every phone.



Figure 5. English: predicted duration (E1, E2a, E2b, E3, E4)

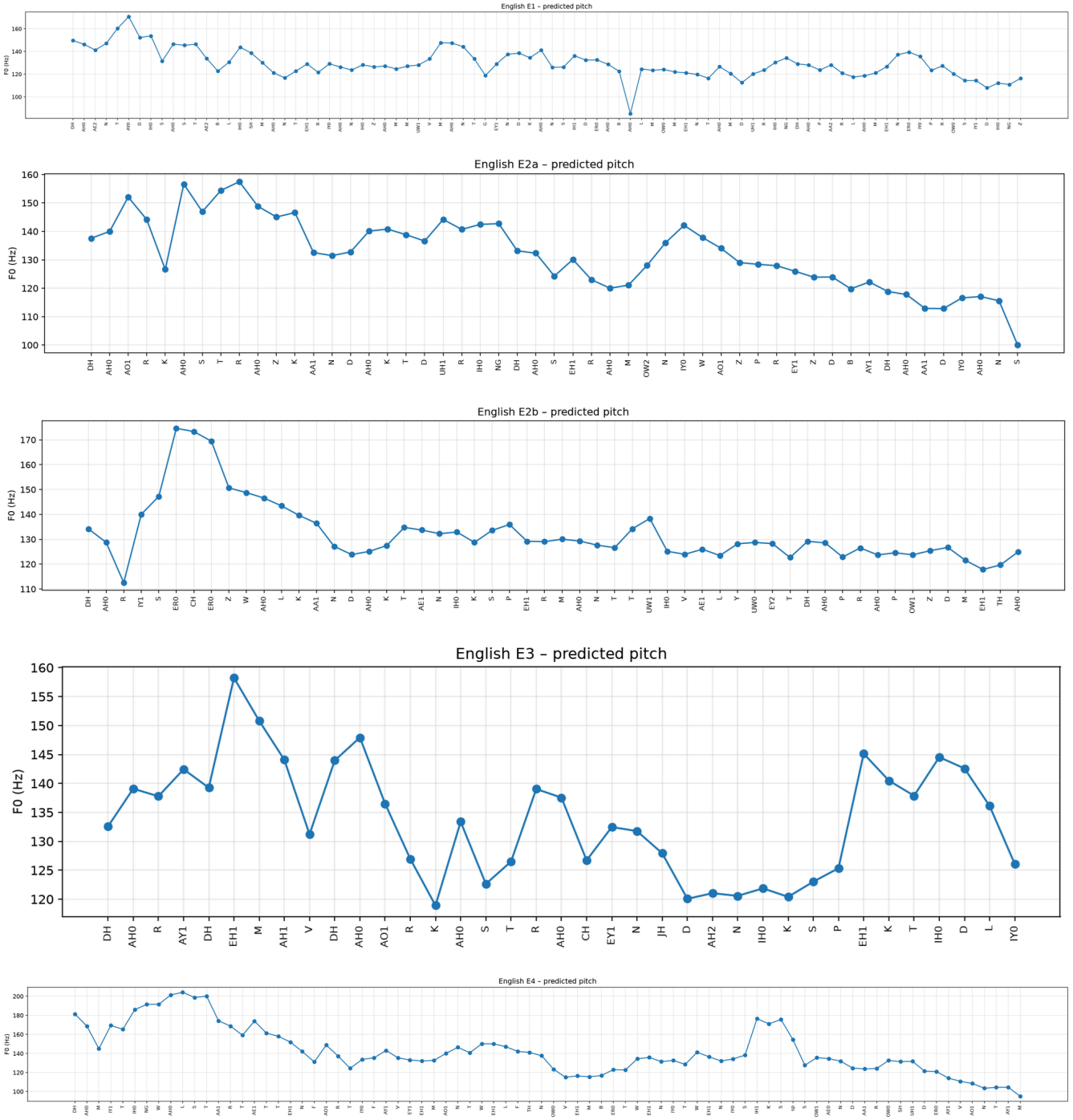


Figure 6. English: predicted pitch (E1, E2a, E2b, E3, E4)

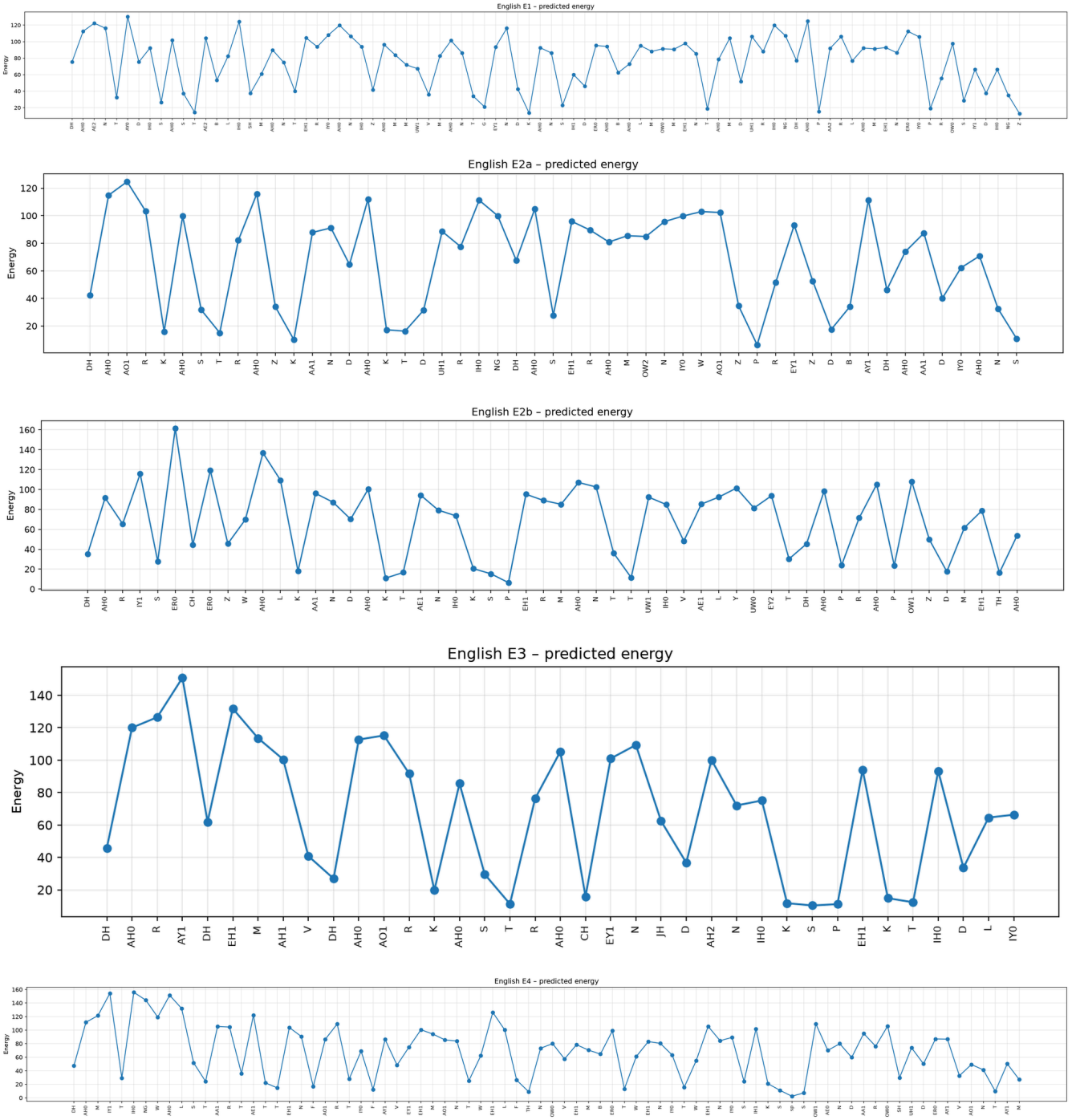


Figure 7. English: predicted energy (E1, E2a, E2b, E3, E4)

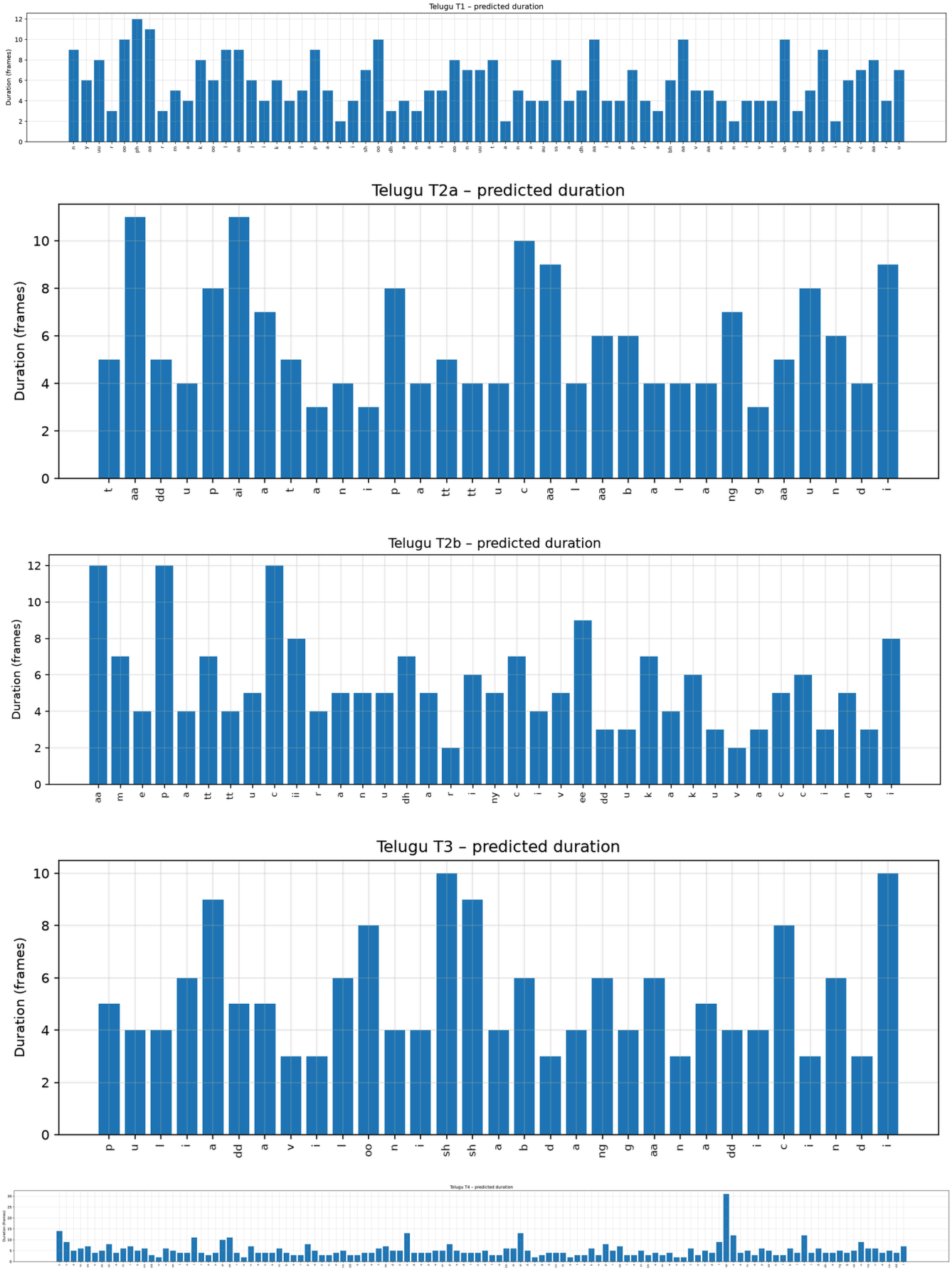


Figure 8. Telugu: predicted duration (T1, T2a, T2b, T3, T4)

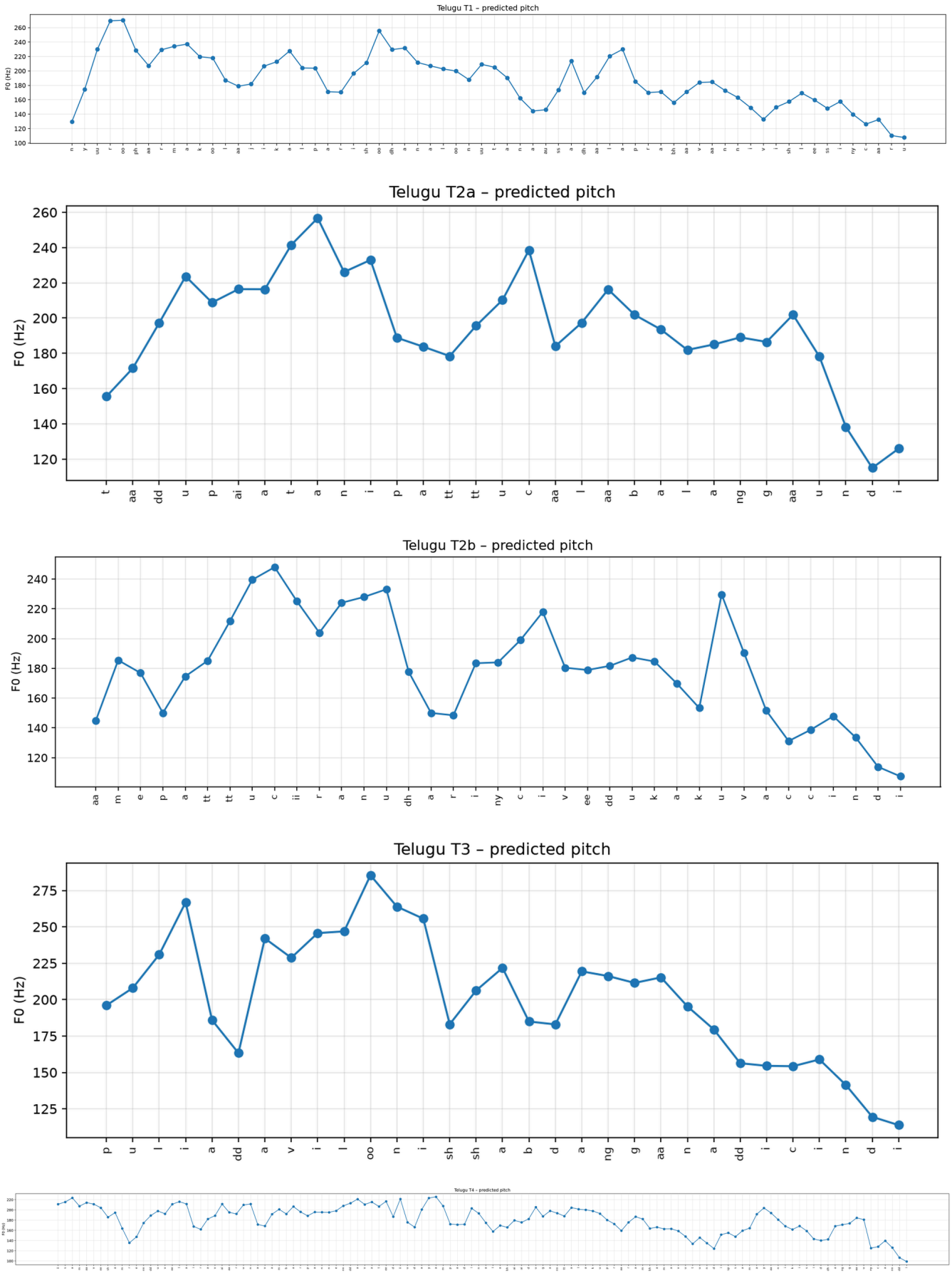


Figure 9. Telugu: predicted pitch (T1, T2a, T2b, T3, T4)

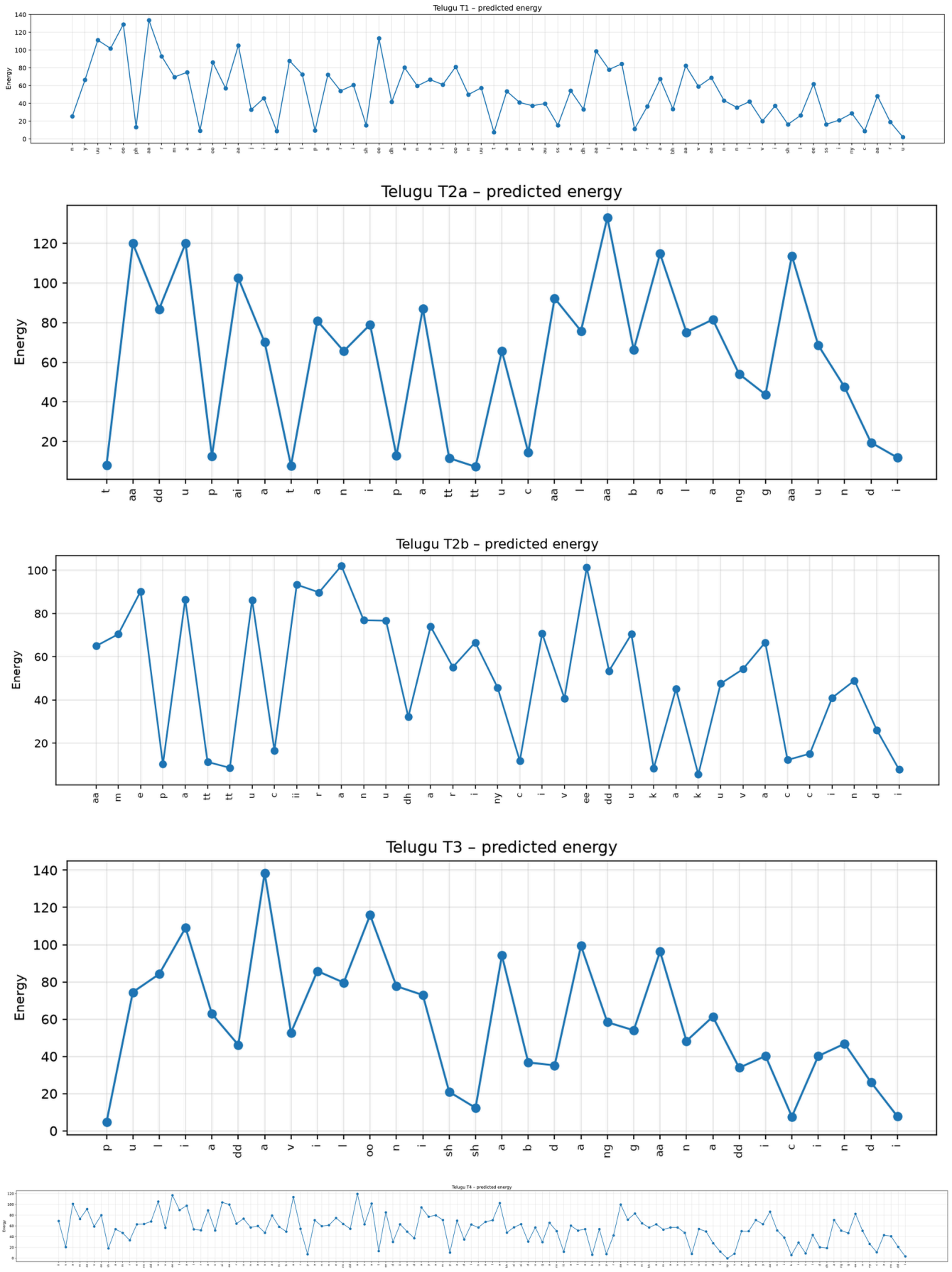


Figure 10. Telugu: predicted energy (T1, T2a, T2b, T3, T4)

How to read the plots. Duration is how long each sound lasts (in milliseconds). Pitch (F0, in hertz) is how high the voice is. Energy is how loud each sound is.

Sentence	Length	Average pitch	Pitch at start → end	Pause at comma
E1	5.0 s	129 Hz	149 → 112 Hz	—
E2a	3.5 s	132 Hz	140 → 112 Hz	—
E2b	3.3 s	132 Hz	133 → 122 Hz	—
E3	2.7 s	134 Hz	138 → 137 Hz	—
E4	6.0 s	142 Hz	166 → 103 Hz	0.44 s
T1	4.5 s	188 Hz	215 → 123 Hz	—
T2a	2.1 s	195 Hz	191 → 152 Hz	—
T2b	2.4 s	180 Hz	166 → 128 Hz	—
T3	1.9 s	201 Hz	218 → 138 Hz	—
T4	7.4 s	181 Hz	214 → 120 Hz	0.36 s

What all sentences have in common

- **Pitch falls towards the end of each sentence,** as it does in natural speech. The fall is larger in Telugu than in English.
- **The last sounds of a phrase are stretched,** as people naturally do. For example, the final "-ly" of "unexpectedly" lasts 0.27 s.
- **Vowels are loud and consonants are quiet** in the energy plots. Double consonants in Telugu (like "ṭṭ") show up as a long, almost silent gap.
- **Pauses appear only at commas** (E4 and T4). Spaces between words never produce a pause.
- **The Telugu pitch is high for a male voice** (180–200 Hz on average). This could be his speaking style, or errors in the pitch measurements used for training. The second would also explain why Telugu pitch was harder to predict (Section 6).

English

E1: a long, rare word. "Antidisestablishmentarianism" is read smoothly as one word, without pauses. The pitch is highest near the start ("an-**ti**-") instead of on the stressed syllable "-**men**-". The model has not learned the stress of such a rare word and just lets the pitch fall gradually.

E2: the same spelling, two meanings ("conduct"). In E2a "conduct" is a noun (CON-duct); in E2b it is a verb (con-DUCT). Our system gives both the noun pronunciation, so E2b is wrong. Interestingly, the model still changes the timing slightly based on the sentence: in E2b the first vowel is shorter and the second longer, hinting at the verb's stress. But because the sounds themselves are the noun's, listeners hear the noun.

E3: a word split by a space ("rhyth m"). The space turns it into two words. "rhyth" is not in the dictionary and is guessed as "rye-th", and the lone "m" is read as the letter "em". That "em" gets the highest pitch in the sentence, as if it were an important word. There is no pause between the two parts.

E4: numbers and a Telugu word. The time, date and year are all read correctly as words. The comma produces a natural 0.44 s pause, and the word "so" after it is drawn out. The Telugu word "andaroo" is read with English sounds, evenly timed and without emphasis, because the English system has no knowledge of Telugu.

Telugu

T1: a long English loanword. న్యూరోఫార్మకోలాజికల్ (neuropharmacological) is read as one unit. The pitch jumps high on the second syllable and stays high across the word. The longest sound is "ph": our rules read ఫ as an aspirated "p" (as in native Telugu words), but in this English loanword it should be "f".

T2: the same word, two meanings (పట్టు: "grip" and "silk"). Both get exactly the same sounds. Unlike "conduct", this is correct, because both meanings are pronounced the same in Telugu. Only the timing changes a little with the sentence.

T3: a word split by a space ("అడ విలో"). Our Telugu rules work letter by letter and ignore spaces, so the split word gets exactly the same sounds as the normal word అడవిలో. The plots show no pause or break at the space.

T4: numbers. "2026", "12వ" and "10:45" are all read correctly as Telugu words, and the comma produces a 0.36 s pause. The first word ఈ has a long vowel and is correctly the longest sound in the sentence. The number words are spoken quickly, and some final vowels are almost swallowed.

9. Comparing English and Telugu

Telugu was the easier language to model: one consistent speaker, clean studio recordings, and a script that is written almost exactly as it sounds. English was harder: 115 speakers with only a few minutes each, and lower-quality recordings. The losses and the sound quality reflect this.

	English	Telugu
Recordings	115 speakers, a few minutes each, some background noise	1 speaker, studio quality
How text becomes phones	dictionary plus a model that guesses from spelling	simple rules
Validation mel loss	0.74	0.61
Validation duration loss	0.23	0.07
Sound	understandable, slightly muffled and noisy	understandable, cleaner

What went wrong

- **Words with two pronunciations (E2).** The system always gives "conduct" its noun pronunciation, because it looks at each word on its own and ignores the sentence.
- **Words from another language (E4, T1).** The English system reads the Telugu word "andaroo" with English sounds, and the Telugu rules read the "ph" of an English loanword as an aspirated "p" instead of "f".
- **Spaces inside words (E3, T3).** In English the space creates two wrongly pronounced words. In Telugu the space is simply ignored.
- **Limited data.** About 5 hours per language is little for TTS (common datasets have about 24 hours), which is why both models started memorizing early. The English recordings also lack high-frequency detail, which no model can restore.
- **Limited listening test.** Only two listeners, and no native Telugu speaker to judge pronunciation.

How it could be improved

1. **Start from a model already trained on a large dataset** instead of from scratch. A FastSpeech 2 trained on 24 hours of English (LJSpeech) is available. Its later layers, which learn what speech sounds like in general, would help both languages. Its knowledge of English rhythm and sounds would help English but not Telugu, so those parts would need retraining for Telugu. We did not do this because it would have meant redoing all training before the deadline.
2. **Use the sentence to choose pronunciations,** for example by checking whether "conduct" is used as a noun or a verb.

3. **Recognize words from other languages** and give them the right pronunciation (Telugu words in English text, English loanwords in Telugu text).
4. **Stop training earlier or make the model smaller** to reduce memorizing.
5. **Use cleaner English recordings,** or remove noise from Svarah before training.
6. **Run a larger listening test** with native Telugu speakers, scoring both naturalness and pronunciation.

10. How to reproduce the results

Everything needed is in the GitHub repository. Exact software versions are listed in `requirements.txt`.

Item	Detail
Software	Python 3.11, PyTorch 2.11 (CUDA 12.8), Montreal Forced Aligner 3.4.3
Settings	`English/config.yaml`, `Telugu/config.yaml`
Train/validation split	`data/splits/` (fixed seed 42)
Trained models	`English/checkpoints/`, `Telugu/checkpoints/`: FastSpeech 2 at step 25,000 and the retrained vocoder at step 20,000
Hardware used	one NVIDIA H100 GPU. Free GPUs also work: on Kaggle or Google Colab (T4), training takes about 5–7 hours per language. Generating speech with the trained models also runs on a CPU.

```
# setup
pip install -r requirements.txt
python -c "import nltk; [nltk.download(p) for p in
['cmudict','averaged_perceptron_tagger_eng']]"
git lfs pull                          # downloads the trained models

# generate speech with the trained models
python English/inference.py           # the 5 test sentences ->
Results/English/
python Telugu/inference.py
python English/inference.py --text "Hello world." --id demo --out my_outputs

# train from scratch
bash scripts/align.sh                 # find phone timings (alignment)
python English/preprocess.py          # extract features and apply the split
python Telugu/preprocess.py
python English/train.py               # 100,000 steps; resume with --
restore_step N
python Telugu/train.py
python scripts/make_gta.py            # prepare data to retrain the vocoder
(see README)
```

References

1. Y. Ren, C. Hu, X. Tan, T. Qin, S. Zhao, Z. Zhao and T.-Y. Liu. "FastSpeech 2: Fast and High-Quality End-to-End Text to Speech." ICLR 2021. arXiv:2006.04558.
2. J. Kong, J. Kim and J. Bae. "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis." NeurIPS 2020. arXiv:2010.05646. (Section on fine-tuning the vocoder on mel-spectrograms predicted by an acoustic model.)
3. J. Shen, R. Pang, R. J. Weiss et al. "Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions" (Tacotron 2). ICASSP 2018. arXiv:1712.05884. (WaveNet vocoder trained on ground-truth-aligned predicted mels.)